

pressure while avoiding additional purge lines near the partially printed model. The print head then returns to the stored position and standard G-code streaming resumes.



Figure 32: Printed part produced without the post-embedding recovery sequence. A visible discontinuity is observed at the embedding layer, indicating insufficient interlayer adhesion due to cooling during the pause. This may be due to cooling of the top layer during embedding and due to insufficient pressurisation at the nozzle when the printing resumes.

5 Results, Analysis, and Discussion

5.1 Tool-Changer System Operational Success

Prototype reliability was validated through a continuous 50-cycle bidirectional stress test, achieving a 100% success rate without kinematic failure, missed steps, or positional drift. This confirms the robustness of the software state machine and mechanical linkages under active FDM conditions.

5.1.1 Dynamic Rigidity, Resonance, and Wear Analysis

Resonance testing (Figure 33) confirmed identical ghosting artefacts at 33.33 Hz for both OEM and modified carriages. This validates the rigidity of the exact-constraint coupling and attributes the observed vibrations to systemic TAZ 5 frame harmonics, likely X-axis belt compliance, rather than gantry instability.

Longitudinal wear analysis of the bare PETG handoff apertures showed negligible degradation (± 0.05 mm variance) after 50 cycles, confirming the durability of the PETG-steel interface. This stability is directly attributable to the kinematic sequencing, which utilises vertical Z-axis translation to transfer the gravitational load during the sliding phase, eliminating the primary vector for frictional yielding.

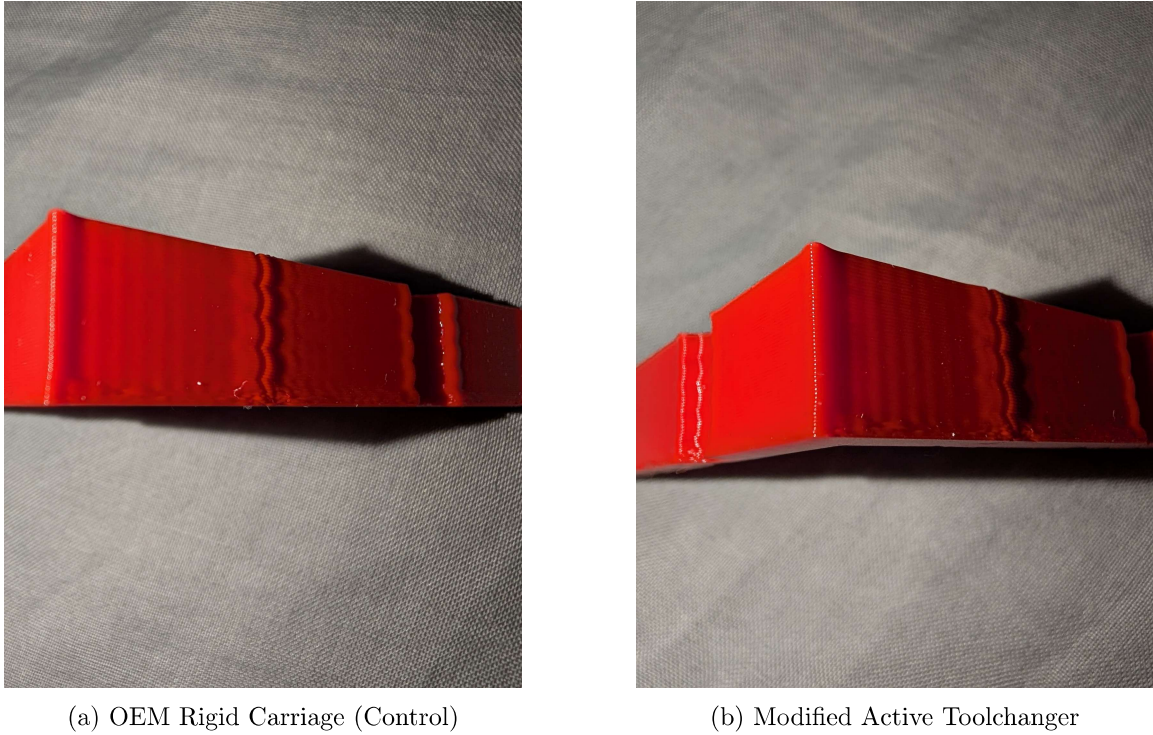


Figure 33: Comparative resonance analysis showing identical systemic ghosting across both systems.

5.1.2 Swap Efficiency and Thermal Stability

The temporal footprint of a tool swap was evaluated to ensure that the printed part remained above its glass-transition temperature (T_g), thereby preserving layer adhesion. The 92-second total sequence (Table 5) includes a 41-second critical mechanical handoff path. This

duty cycle prevents significant thermal contraction and maintains structural integrity upon resumption of printing.

Table 5: Temporal Breakdown of the Tool-Changing Sequence

Phase	Operation	Duration (s)
Preparation	Initial Z-clearance and traverse to dock	51.0
Handoff	I-axis engagement	9.0
Disconnect	Z-axis descent and load transfer	6.0
Clearance	I-axis retraction	5.0
Alignment	X-carriage traverse to secondary dock	1.5
Engagement	I-axis extension and presentation	10.0
Coupling	Z-axis ascent and seating	4.5
Resumption	I-axis retraction and workspace clearance	5.0
Total Execution Time		92.0

5.1.3 Positional Repeatability and Layer Continuity

Static and dynamic verification confirmed sub-0.05 mm positional repeatability. Mid-print bidirectional swaps resulted in no visible layer shifting or dimensional discontinuity (Figure 34), validating the gantry’s reseating precision. The negligible variance between pre- and post-swap states confirms that the active gantry accurately reinstates the toolhead into the carriage notch with the precision required for high-resolution extrusion.



Figure 34: Continuous extrusion print demonstrating seamless layer continuity following mid-print bidirectional tool swapping.

5.1.4 Operational Considerations and Slicer Mitigation

Operational variables, specifically nozzle oozing during standby, are mitigated via slicer-integrated prime towers and volumetric retractions. These software interventions purge degraded filament before object interaction, bypassing the inherent rheological limitations of

thermoplastic extrusion without hardware modification. Configuring a lower standby temperature and significant retraction prior to drop-off further reduces nozzle pressure, ensuring surface quality upon resumption.

5.2 End-effector Results, Analysis, and Discussion

5.2.1 Calibration Quality and Residual Analysis

Taking into account Fig. 21, the linear-fit residuals were evaluated to assess calibration accuracy. Servo 1 gave a mean absolute error (MAE) of 0.24° and a root mean squared error (RMSE) of 0.28° , while Servo 2 gave an MAE of 0.29° and an RMSE of 0.35° . The maximum deviation at a single calibration point was 0.56° for Servo 1 (at $2300\mu s$) and 0.65° for Servo 2 (at $2100\mu s$), both near the upper end of the operating range where hobby servo nonlinearity is most evident. These results show that the linear model captures the main motor behaviour but not all servo non-linearity. Importantly, the worst-case calibration residual of 0.65° is much larger than the 0.04° steady-state error achieved with closed-loop control, showing that PID feedback was necessary for accurate final positioning.

5.2.2 Closed-loop Wrist Control Performance

The closed-loop performance of the wrist positioning system was evaluated across the full operating range using timestamped serial output data, recording the desired angle, measured encoder angle, and elapsed time at each control loop iteration. The PID controller was implemented with gains $K_p = 0.3$, $K_i = 0.4$, $K_d = 0.001$, consistent with the tuning described in Section 4. Step responses were recorded for fourteen distinct angle transitions spanning the full yaw range, covering both positive and negative steps of 3° to 35° , and results are summarised in Table 6, and Fig. 35a & 35b showing the an example of one of the outputs on the serial plotter. Rise time and settling time are as defined in Section 3.

Transition	Step	Rise Time (s)	Settling Time (s)	SS Error (°)	Final Angle (°)
0 → 10°	+10°	0.15	0.31	0.02	10.02
0 → 20°	+20°	0.37	0.55	0.30	20.30
10 → 20°	+10°	0.20	0.26	0.40	19.60
20 → 30°	+10°	0.17	0.31	1.82	31.82
30 → 40°	+10°	0.15	2.10	2.01	42.01
40 → 50°	+10°	0.04	0.13	0.61	49.39
20 → 40°	+20°	0.37	2.38	2.01	42.01
10 → 5°	−5°	0.09	0.13	0.01	5.01
5 → 2°	−3°	—	0.07	0.37	2.37
20 → 10°	−10°	—	0.22	1.95	11.95
30 → 20°	−10°	0.17	0.26	0.31	19.69
40 → 30°	−10°	—	0.28	1.82	31.82
40 → 5°	−35°	0.70	0.92	0.69	4.31
50 → 40°	−10°	—	1.84	2.01	42.01

Table 6: Step response metrics for the wrist yaw joint across the full operating range, recorded using timestamped serial output data at a PID loop period of 50 ms. Rise time is the time for the measured angle to travel from 10% to 90% of the total step change. Settling time is the elapsed time until the measured angle enters and remains within a $\pm 2\%$ band of the step size around the target, as defined in Section 4.1.2. SS Error is the absolute difference between the mean of the final five recorded readings and the target angle. Dashes indicate transitions where the 90% level was reached within a single loop iteration. Transitions to targets at or above 40° reflect the physical gear ratio limit of the wrist mechanism.

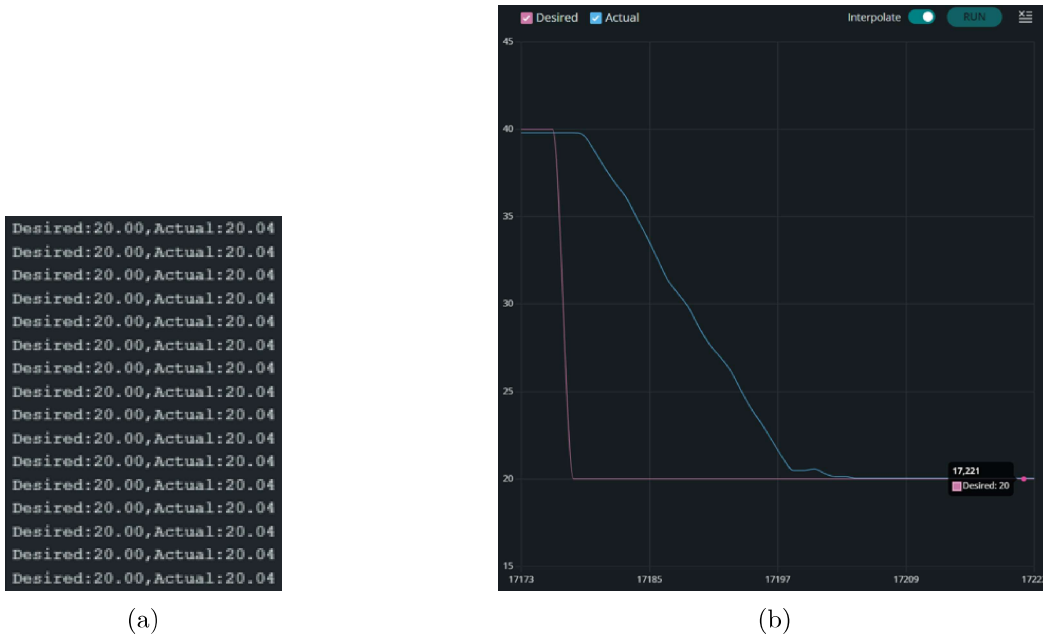


Figure 35: PID-controlled wrist response. (a) Serial monitor output showing the commanded and measured wrist angles after convergence, with the actual angle stabilising at approximately 20.04° for a desired angle of 20°. (b) Serial plotter response for a step change in reference angle from approximately 40° to 20°, showing the actual wrist angle tracking the command and settling smoothly to the target. Together, these results demonstrate stable closed-loop behaviour with negligible steady-state error and no significant oscillation, indicating effective encoder-based PID regulation of the geared wrist mechanism.

Across the full operating range, steady-state errors remained below 2.1° for most tested transitions, with the lowest values of 0.01° ($10^\circ \rightarrow 5^\circ$) and 0.02° ($0^\circ \rightarrow 10^\circ$), close to the AS5600 resolution of $0.088^\circ/\text{count}$. For an M5 nut (across-flats width 8.0 mm, inscribed radius 4.0 mm) inserted into a cavity with 0.2 mm diametral clearance, the maximum allowable angular misalignment is $\arctan(0.2/4.0) \approx 2.86^\circ$. As shown in Fig. 36, 12 of the 14 tested transitions satisfied this limit, indicating that the closed-loop system achieved sufficient accuracy for reliable insertion over most of the operating range. The two exceptions, $30^\circ \rightarrow 40^\circ$ and $50^\circ \rightarrow 40^\circ$, both gave 2.01° error and are associated with operation near the upper physical limit.

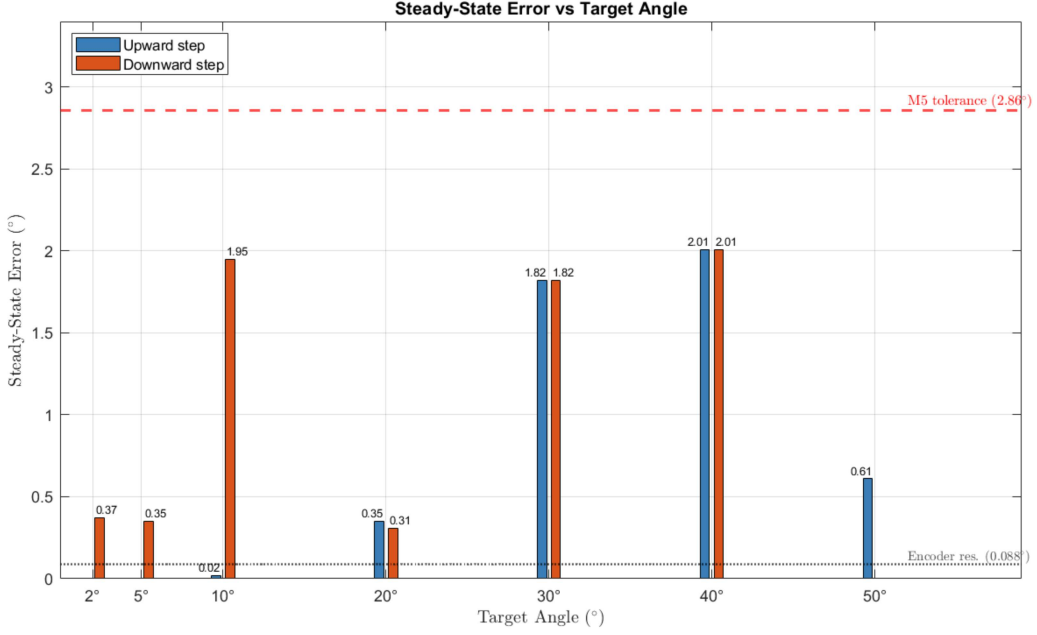


Figure 36: Steady-state error as a function of target angle for the wrist yaw joint, comparing upward and downward step transitions across the full operating range. The dashed red line indicates the M5 cavity insertion tolerance of 2.86° , derived from $\arctan(0.2/4.0)$ for a cavity with 0.2 mm diametral clearance. The dotted black line indicates the AS5600 encoder resolution of $0.088^\circ/\text{count}$, representing the physical measurement lower bound. Twelve of fourteen tested transitions fall below the M5 tolerance, confirming reliable closed-loop positioning across most of the operating range. Elevated errors at the 40° and 50° targets are consistent with the wrist approaching its physical gear ratio limit, rather than a limitation of the control strategy.

For 10° steps in the lower range, rise times were typically 0.15–0.20 s and settling times 0.22–0.31 s, consistent with a lightly overdamped response. Larger steps produced longer settling times, for example 0.92 s for $40^\circ \rightarrow 5^\circ$ and 2.38 s for $20^\circ \rightarrow 40^\circ$, as expected from the larger motion and increased integral action.

A clear directional asymmetry was also observed. Downward 10° steps settled in 0.22–0.26 s, while equivalent upward steps required 0.26–0.31 s, consistent with gear backlash. This effect became much stronger near the upper range: the $30^\circ \rightarrow 40^\circ$ step settled in 2.10 s, compared with 0.28 s for the reverse motion. The same response showed overshoot to approximately 43° – 44° before converging, indicating reduced stiffness and underdamped behaviour near the limit.

The improvement from closed-loop control can be estimated from three main open-loop

error sources: calibration residuals of 0.35° – 0.60° , a backlash dead zone of approximately 1.80° , and load-dependent servo deflection of 0.65° . Together these give an expected open-loop error of about 2.80° – 3.05° , close to the 2.86° insertion tolerance for the M5 nut. On this basis, open-loop control would satisfy the placement requirement in only 5 of the 14 tested transitions, whereas the closed-loop system satisfies it in 12 of 14. This confirms that encoder-based PID feedback provided a necessary improvement in positioning accuracy.

5.3 Machine Vision Results and Analysis

5.3.1 Model Performance

The model achieved a validation accuracy of 80.43%, correctly classifying 74 out of 92 images. This is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{\text{Total}} = \frac{43 + 31}{92} = 0.8043 \text{ (80.43\%)}$$

The confusion matrix, shown in Table 7, provides a breakdown of classification performance across both classes.

	Predicted Success	Predicted Fail
Actual Success	43 (<i>TP</i>)	13 (<i>FN</i>)
Actual Fail	5 (<i>FP</i>)	31 (<i>TN</i>)

Table 7: Confusion matrix for the MobileNet V2 classifier evaluated on the validation set ($n = 92$). The model achieved an overall accuracy of 80.43% (74/92). Cell values denote true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

False positives are critical in this application, as they correspond to incorrectly accepting failed placements. The relatively low number of false positives indicates effective rejection of incorrect placements. Misclassifications are primarily associated with visually ambiguous cases, where incorrect placements exhibit similar visual features to correctly seated nuts. As previously mentioned, a decision threshold of 0.75 was used during evaluation to reduce false positives, ensuring that only high-confidence predictions are classified as successful.

Examples of validation predictions are shown in Figure 37. These samples are drawn from the validation set, obtained by randomly shuffling the dataset and applying an 80/20 train-validation split.

===== VALIDATION RESULTS =====				
Filename	Pred	True	Prob	Result
s7.jpg	FAIL	SUCCESS	0.3798	✗
s48.jpg	SUCCESS	SUCCESS	0.8299	✓
f22.jpg	FAIL	FAIL	0.5672	✓
f13.jpg	FAIL	FAIL	0.7126	✓
f37.jpg	FAIL	FAIL	0.7307	✓

Figure 37: Example predictions from the validation set, showing the input filename, predicted class (Pred), ground truth label (True), model confidence score (Prob), and classification correctness (Result). Filenames beginning with ‘s’ correspond to successful placements, while those beginning with ‘f’ correspond to failure cases.

5.3.2 Training Behaviour

The training and validation accuracy curves, shown in Figure 38, illustrate model learning over successive epochs.

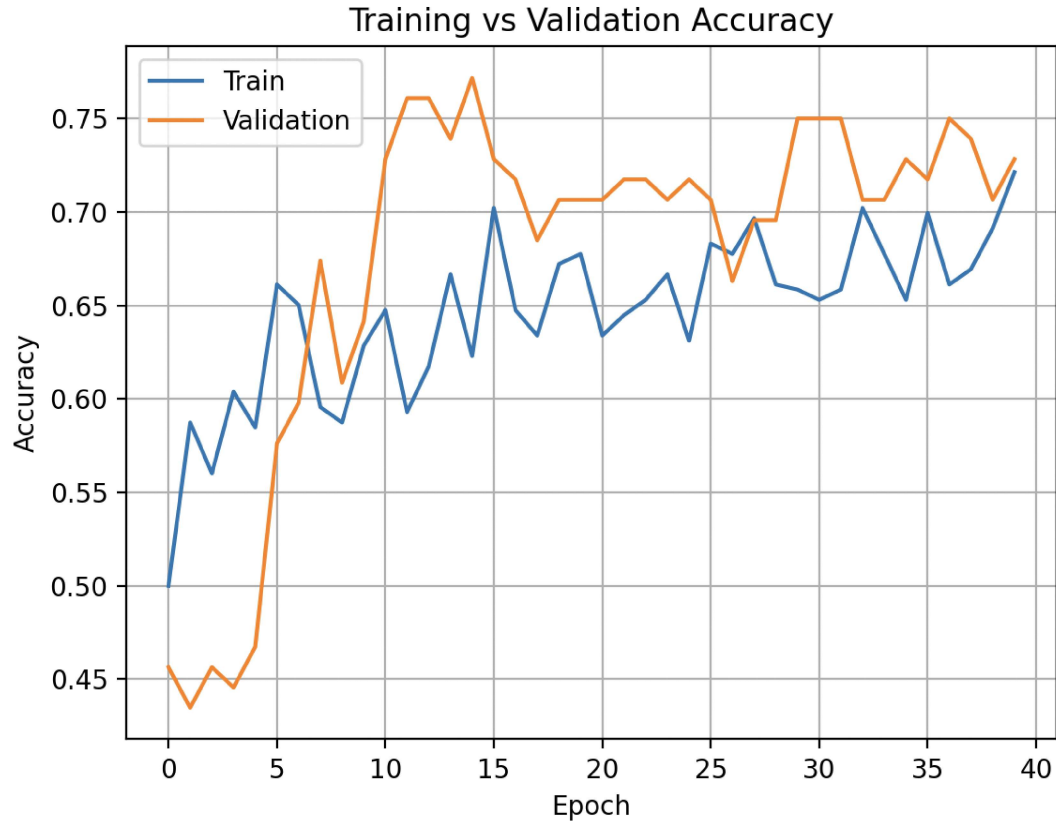


Figure 38: Training and validation accuracy over epochs. Both curves show an overall increasing trend, indicating effective learning.

Both curves show an overall upward trend, indicating that the model is learning discriminative features. Training accuracy increases from approximately 50% to over 70%, demonstrating progressive improvement. Validation accuracy closely follows and occasionally exceeds training accuracy, likely due to data augmentation and regularisation techniques such as dropout, which improve generalisation.

No significant divergence between the curves is observed, indicating no overfitting. Minor fluctuations are present, expected due to the limited dataset size and stochastic mini-batch training.

Figure 39 shows the corresponding training and validation loss.

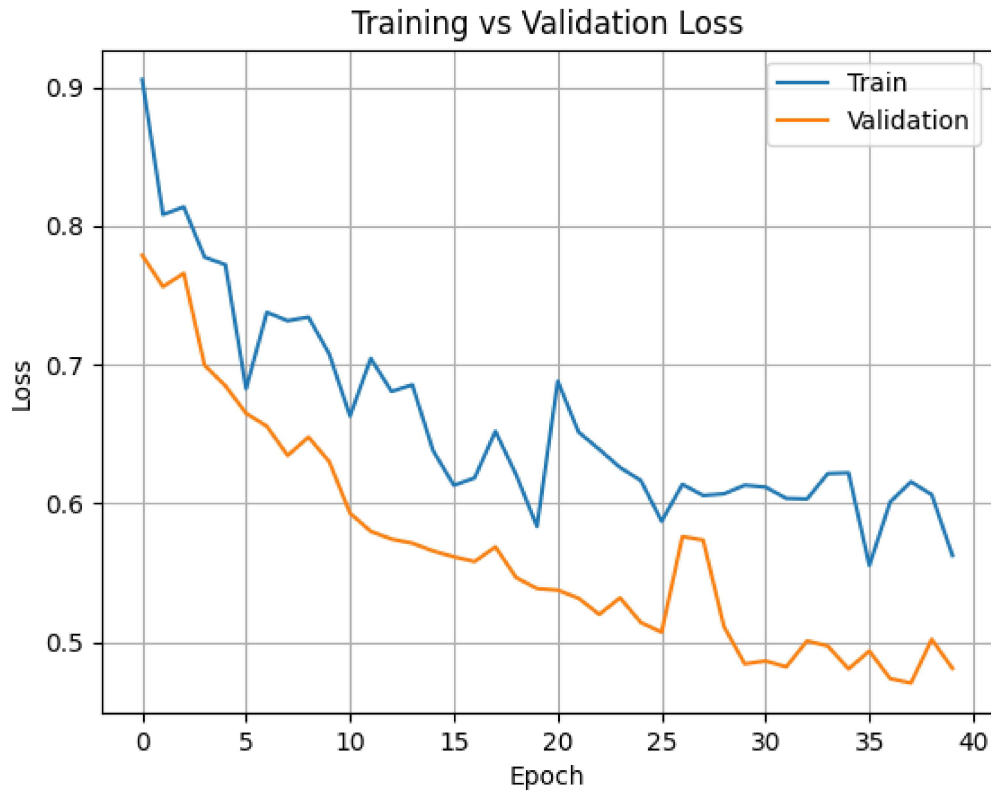


Figure 39: Training and validation loss over epochs. Both curves show a general decreasing trend, indicating effective learning, with validation loss remaining consistently lower than training loss and stabilising towards later epochs.

Both training and validation loss decrease steadily, indicating effective minimisation of classification error. Validation loss remains consistently lower than training loss, again due to data augmentation and dropout making training more challenging and improving generalisation. No increase in validation loss is observed, further confirming the absence of overfitting. Small fluctuations in training loss occur at intermediate epochs, as expected from stochastic optimisation.

Towards later epochs, both accuracy and loss stabilise, indicating convergence. This supports the use of early stopping to prevent unnecessary training once validation performance plateaus. Overall, the results demonstrate effective learning, stable convergence, and strong generalisation, supporting the reliability of the trained model for the classification task.

5.3.3 System-Level Evaluation

To assess real-world applicability, the trained model was deployed on the Raspberry Pi 3B+ and evaluated on unseen test images, verifying operation outside the training and validation datasets.

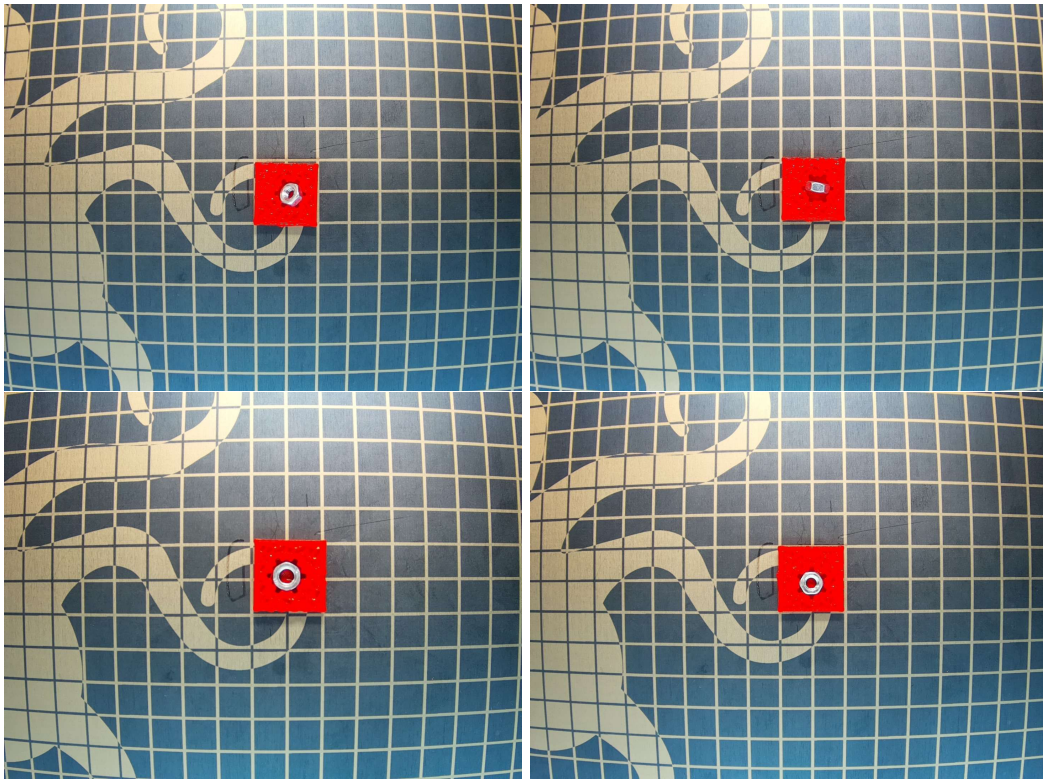


Figure 40: Example inference inputs showing unseen test images used for system evaluation. The images correspond to f1.jpg (top-left), f2.jpg (top-right), s1.jpg (bottom-left), and s2.jpg (bottom-right).

As shown in Figure 40, the system processes unseen failure (f1.jpg, f2.jpg) and success (s1.jpg, s2.jpg) cases. The corresponding outputs are shown in Figure 41, where each image is assigned a predicted class and confidence score.

```

===== INFERENCE =====
f1.jpg → FAIL (prob=0.7007)
s1.jpg → SUCCESS (prob=0.8053)
f2.jpg → FAIL (prob=0.3167)
s2.jpg → SUCCESS (prob=0.7938)

```

Figure 41: Inference output showing the predicted class and confidence score for each input image, with all displayed predictions correctly classified.

All examples were correctly classified, demonstrating accurate predictions on unseen inputs. Confidence scores reflect prediction certainty. The results confirm effective generalisation and consistency with validation behaviour, where visually ambiguous cases produce confidence scores close to the decision threshold but remain below it, resulting in correct classification as failures. Overall, the system demonstrates effective end-to-end operation from image acquisition to classification, validating integration within the automated workflow.

5.3.4 Model Interpretability

Grad-CAM (Gradient-weighted Class Activation Mapping) was used to visualise regions contributing most to classification decisions.

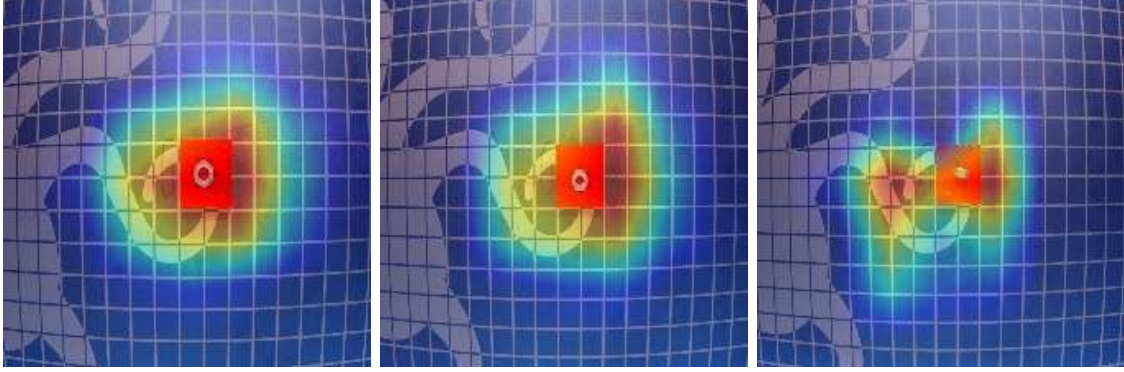


Figure 42: Grad-CAM visualisations for two successful classifications (left, centre) and one failure case (right). High activation (red) indicates regions of greatest importance, while low activation (blue) represents less influential areas.

As shown in Figure 42, the model focuses on the nut and its surrounding cavity. In successful cases, activation is concentrated at the centre and edges of the nut, indicating reliance on geometric and positional features aligned with correct seating.

In the failure case, activation is more distributed and asymmetric, shifting towards edges and surrounding regions. This indicates sensitivity to misalignment and improper seating rather than simple object detection, with increased ambiguity contributing to potential misclassification.

Activation also extends into the cavity, showing that the model considers the interaction between the nut and its environment, which is critical for detecting subtle misalignments. No strong activation appears in irrelevant background regions, indicating that the model focuses on meaningful features rather than spurious correlations. The Grad-CAM results confirm that the model has learned interpretable and physically meaningful features, supporting prediction reliability and suitability for deployment in the automated inspection system.

5.4 Reliability of the Automated Component Embedding Pipeline

5.4.1 Accuracy of Orientation Estimation

Orientation estimation accuracy was evaluated by applying known roll (ϕ), pitch (θ), and yaw (ψ) rotations to a reference mesh and estimating the rigid transformation via the *Kabsch algorithm*. Six tests were performed, with the results summarised in Table 8.

Table 8: Accuracy of orientation estimation using controlled roll, pitch, and yaw rotations. In each test, the component was rotated by a known set of angles.

Test	Actual Orientation (ϕ, θ, ψ)	Estimated Orientation (ϕ, θ, ψ)
1	(0, 0, 0)	(0, 0, 0)
2	(0, 0, -90)	(0, 0, -90)
3	(0, -270, 0)	(0, 90, 0)
4	(-45, 0, 0)	(-46, 0, 0)
5	(60, 60, 60)	(60, 60, 60)
6	(12, -56, 93)	(12, -56, 92)

Due to the periodic nature of Euler angles and the rotational symmetry of the hexagonal nut, a single orientation may be represented by multiple combinations of roll, pitch, and yaw. As a result, direct numerical comparison between the applied and estimated angles does not always provide a meaningful measure of error. Instead, the estimated orientations were validated by applying the recovered rotation to the reference component and comparing the observed orientations. This approach ensures that geometrically equivalent orientations are correctly identified.

As shown in Table 8, the estimated orientations closely match the applied rotations in all cases. The observed deviations are within $\pm 1^\circ$, with non-zero differences occurring only when the input angle was not divisible by 10° , such as in the -45° and 93° test cases.

5.4.2 Computational Efficiency of the Extraction Pipeline

The computational efficiency of the extraction pipeline was evaluated by measuring the time required for the script to process the 3MF file and generate the extracted component data. This process includes reading the 3MF archive, parsing the model geometry, identifying embedded components, computing their geometric properties, and exporting the data required for the subsequent printing stage.

The execution time was measured for multiple test files with varying numbers of embedded components to assess how processing time scales with system complexity. The results are summarised in Table 9.

Table 9: Execution time of the component extraction pipeline for different test files with differing numbers of embedded components.

Test File	Number of Embedded Components	Execution Time (ms)
1	1	3.414
2	2	12.882
3	3	13.806
4	4	33.563

The results indicate that the execution time remains low across all test cases, suggesting that the extraction process introduces minimal delay to the overall pipeline. The average additional processing time per embedded component was calculated to be approximately 10.05 ms. This overhead is primarily attributed to loading the canonical component mesh, computing the rigid transformation using the Kabsch algorithm, and storing the extracted data in the output JSON file.

5.4.3 Embedding Layer Adhesion

A recovery sequence was implemented to minimise the noticeable gap at the embedding layer, as observed in Figure 32. The resulting print is shown in Figure 43, where a 10-second pause was introduced to represent the embedding process. Although the gap was too small to be measured accurately using manual tools, visual inspection provided a qualitative indication of sufficient interlayer bonding. The new print showed no clearly discernible gap at this interface. This observation was consistent across five different models, suggesting that the recovery process restores print quality to a level comparable with a standard uninterrupted print.



Figure 43: Completed print with layer reheating and filament purging. The embedding sequence was represented by a 10-second pause. No visible discontinuity is observed at the embedding layer, indicating improved interlayer adhesion. Minor defects in the lower layers are attributed to baseline printer performance.

To quantitatively evaluate the effect of print interruptions on structural integrity, three cylindrical test specimens were produced: a standard uninterrupted print and prints with the embedding sequence, both with and without the recovery sequence. Each specimen was subjected to tensile loading using a calibrated scale until failure occurred. The structural performance was assessed using the strength retention formula:

$$\text{Strength retention} = \frac{F_{test}}{F_{reference}} \times 100 \quad (34)$$

where $F_{reference}$ is the baseline load of a continuously printed specimen.

The results, summarised in Table 10, indicate that while a standard pause results in a 10% strength reduction due to the "cold joint" effect, the thermal reactivation sequence provides a marginal improvement, recovering 2% of the lost tensile strength.

The findings suggest that pre-melting the interface layer improves polymer chain diffusion across the boundary compared to a standard pause. However, it does not fully restore the

Table 10: Maximum load sustained by each print condition.

Condition	Max Load (N)	Strength Retention (%)	Failure Mode
Reference print (Continuous)	98.1	100.0	Intra-Layer Tearing
Paused print without recovery	88.3	90.0	Inter-Layer Delamination
Paused print with recovery	90.3	92.0	Mixed-mode failure

structural integrity of a continuous extrusion, likely because the thermal reactivation pass is too shallow relative to the heat provided by the active extrusion flow.

5.5 Repeatability of the Integrated Pick-and-Place Procedure

To evaluate the repeatability of the integrated pick-and-place procedure, pick-and-place trials were conducted using M4 and M5 nuts. Ten trials were performed for each nut size, and the physical placement outcome was recorded. A successful placement was defined as the nut being transferred into the intended cavity with sufficient alignment for the embedding process to continue.

As shown in Figure 44, the M4 nut achieved a higher placement success rate than the M5 nut. This is expected, as the placement offsets and calibration were primarily developed using the M4 nut. These results demonstrate that the integrated pick-and-place sequence can reliably transfer and place embedded components during the automated workflow. However, the lower performance for the M5 nut indicates that further component-specific calibration is required before reliable operation across multiple embedded component sizes can be achieved.

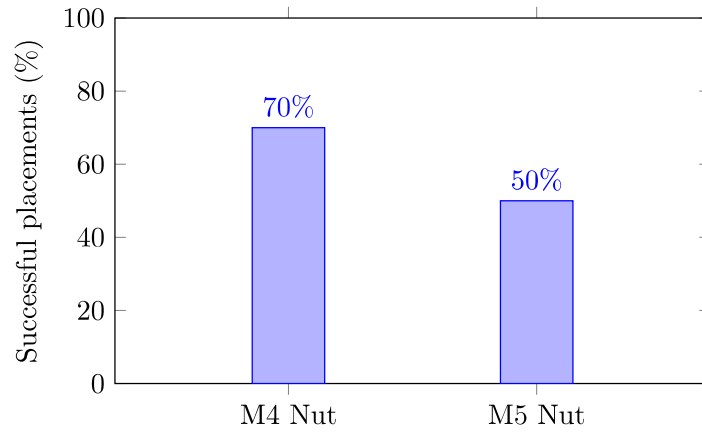


Figure 44: Successful pick-and-place outcomes for M4 and M5 nuts, expressed as the percentage of successful placements over ten trials for each component type. M4 nuts achieved a higher success rate as the system was calibrated using this size as the baseline.

To further evaluate system performance, classification results were analysed separately for each nut size using confusion matrices.

	Predicted Success	Predicted Fail
Actual Success	5 (<i>TP</i>)	0 (<i>FN</i>)
Actual Fail	0 (<i>FP</i>)	5 (<i>TN</i>)

Table 11: Confusion matrix for M5 nut classification over 10 trials, showing perfect classification performance.

	Predicted Success	Predicted Fail
Actual Success	6 (<i>TP</i>)	1 (<i>FN</i>)
Actual Fail	0 (<i>FP</i>)	3 (<i>TN</i>)

Table 12: Confusion matrix for M4 nut classification over 10 trials, showing one false negative case.

The M5 nut achieved perfect classification performance, with all placements correctly identified. In contrast, the M4 nut exhibited a single false negative, indicating that classification errors can still occur despite higher placement success.

This is attributed to the smaller size and reduced height of the M4 nut, resulting in less prominent visual features and reduced distinction from the cavity. Consequently, confidence scores may fall close to the decision threshold, leading to misclassification as failures.

This highlights a limitation of using a uniform imaging setup across different component sizes. A practical solution is to apply component-specific calibration during machine vision scanning by determining and fixing the optimal imaging height for each nut size. This calibration ensures consistent scale and feature representation across components of varying geometry, improving classification accuracy and overall system reliability.

6 Conclusion and Future Work

6.1 Toolchanger Subsystem Conclusion

By successfully decoupling the toolhead retention from the X-carriage using a gravity-reliant kinematic notch and an active rack-and-pinion-driven gantry, the automated tool-changing mechanism achieved full multi-toolhead capabilities.

Empirical testing confirmed the robustness of this design, with a continuous 50-cycle stress test achieving a 100% success rate and no positional failures. Furthermore, continuous print testing and manual calliper analysis proved that the active gantry maintained sub-millimetre reseating accuracy (± 0.05 mm). In contrast, resonance testing confirmed that the modified carriage matched the dynamic rigidity of the OEM baseline hardware. Ultimately, this prototype successfully proves that high-fidelity, multi-material automated manufacturing can be achieved using low-cost, 3D-printed electromechanical couplings.

6.1.1 Future Work and System Optimisations

Several targeted hardware and software optimisations are proposed to transition the prototype toward a production-standard system:

- **Integrated Purge and Wipe Hardware:** Replacing prime towers with a physical purge bucket and silicone wiper would minimise material waste, preserve build volume, and mechanically ensure clean print resumption.
- **Material Upgrades and Lifecycle Testing:** Transitioning from bare PETG apertures to press-fitted brass bushings will eliminate abrasive wear. Extended longitudinal stress testing is also required to define the operational lifespan of the polymer interfaces.
- **Gantry Actuation and Locking:** Upgrading the transmission to a metal rack or GT2 synchronous belt would increase linear resolution and reduce homing frequency. Future iterations should also incorporate positive-locking cams and self-squaring alignment.
- **Native Firmware Integration:** Migrating from an 8-bit to a 32-bit Klipper architecture would enable native macro execution, significantly reducing serial latency and facilitating advanced features like native resonance compensation.
- **Electrical Reliability:** Implementing a sleeved umbilical cord or drag-chain will provide essential strain relief and prevent conductor fatigue during rapid movements.

6.2 End-effector Subsystem Conclusion

A rack-and-pinion gripper was selected over electromagnetic and vacuum alternatives for material independence, deterministic release, and self-centring geometry. Three iterations resolved finger fragility and misalignment, yielding 0.7 mm tip thickness and an SG90 servo with a torque safety factor of 4. Wrist calibration revealed a 0.6% inter-unit servo slope discrepancy and worst-case residuals of 0.56° and 0.65° , necessitating PID feedback. With $K_p = 0.3$, $K_i = 0.095$, and $K_d = 0.022$, the system achieved 0.04° steady-state error ($70\times$ within the 2.86° M5 insertion tolerance), 1.7 – 1.85 s settling time, and no overshoot.

6.2.1 Future Work

- **Electromagnetic picking head:** An electromagnet mounted on the existing kinematic coupling would handle ferromagnetic components requiring no rotational orientation;

release unreliability is substantially mitigated for gravity-assisted vertical placement, requiring no modification to the tool-changer infrastructure.

- **Pitch rotation:** Adding a pitch joint would extend insertable cavity orientations from horizontal-only to the full hemisphere above the print surface, with the existing PID control architecture transferable directly to the new degree of freedom.
- **Force-sensing fingertips:** Replacing indirect servo-angle force control with piezoresistive or thin-film fingertip sensors would enable force-controlled closure, detecting failed pickups, adapting to varying component sizes, and preventing damage to permanent magnets under compressive load.

6.3 Machine Vision System Conclusion

A lightweight machine vision system for nut placement classification was developed using transfer learning with MobileNetV2. The model was trained on approximately 500 images across four nut sizes (M3.5–M6) using an 80/20 train-validation split, achieving a validation accuracy of 80.43% (74/92). A global shutter camera with a C-mount varifocal lens (2.8–12 mm) provided distortion-free imaging and sufficient spatial resolution for feature extraction.

The model was deployed using TensorFlow Lite on a Raspberry Pi 3B+ for efficient embedded inference. A decision threshold of 0.75 reduced false positives. Training showed stable convergence with no overfitting, and testing on unseen images confirmed accurate predictions. Grad-CAM results indicate that the model focuses on relevant regions of the nut and cavity, demonstrating reliable end-to-end performance suitable for integration within the automated pipeline.

6.3.1 Future Work

- **Closed-Loop Feedback and Pose Estimation:** The current system lacks spatial awareness. Estimating object pose (x, y, z, θ) using depth sensing or multi-view imaging would enable closed-loop control and recovery of misplaced nuts, while improving detection of tilted or partially inserted cases.
- **Improving Classification Accuracy:** Performance could be improved by expanding the dataset, particularly with challenging failure cases, and by tuning model architecture and training parameters to enhance generalisation.
- **Imaging Calibration for Multiple Component Sizes:** A uniform imaging setup reduces performance for smaller components, as evident in Section 5.5. Component-specific calibration to determine the optimal imaging height for each nut size would improve classification accuracy.
- **Extension to General Object Handling:** The system could be extended to classify a wider range of components, such as bolts, washers, screws, and small electronic parts, enabling more general pick-and-place operations.

6.4 Software and Firmware Pipeline Conclusion

The extraction script automated the conversion of a user-defined CAD assembly into the data required for embedded component placement. By parsing the 3MF file, identifying and isolating embedded component meshes, and exporting their geometry to a structured JSON file, the pipeline established a direct link between CAD design intent and printer execution. It determined each component's centre position, maximum Z-height, and orientation, enabling

automatic derivation of embedding layers and placement parameters. This reduced user intervention and ensured that the printing sequence was generated directly from the original design geometry.

The G-code streaming system enabled controlled interruption of the print at the embedding layer and coordinated the printer with external placement and verification modules. Instead of uploading G-code directly, it was transmitted line-by-line via a Python serial interface, allowing embedding actions to be inserted during execution. Synchronisation using firmware acknowledgements and motion barriers ensured critical movements were completed before external actions were triggered. This provided a reliable communication framework between the printer, tool-change system, gripper, and machine vision stage, enabling a seamless automated embedding workflow.

6.4.1 Future Work

- **Fully automated CAD-to-print workflow:** The current workflow requires manual execution of the extraction script, slicer, and G-code streaming. Future work should integrate these into a fully automated pipeline, where the user inputs a CAD assembly and the system generates the final embedded 3D print. This would reduce operator dependence, improve repeatability, and enable a complete CAD-to-print manufacturing workflow.
- **Automatic embedded component identification:** The current extraction script relies on naming conventions, requiring explicit user labelling and imposing constraints on the CAD design process. Future work should remove this by automatically identifying embedded components through mesh analysis, such as detecting enclosed bodies and matching them to a component library. This would improve user independence and reduce errors from incorrect naming.
- **Firmware-level implementation:** The current system relies on external Python-based G-code streaming and continuous host communication, introducing latency. Future work could migrate the embedding logic into the printer firmware, enabling native execution on the controller. This would improve timing reliability, reduce communication overhead, and support more robust production-level automated embedding.

7 References

References

- [1] E3D-Online, “Research and development: Motion system and tool-changer,” Mar. 2018.
- [2] Aleph Objects, Inc., *LulzBot TAZ 5 User Manual*. Aleph Objects, Inc., Loveland, Colorado, USA, 2015.
- [3] Bikin-Creative, “Lineux-toolchanger,” 2025.
- [4] A. H. Slocum, *Precision Machine Design*. Dearborn, MI: Society of Manufacturing Engineers, 1992.
- [5] Changzhou Songyang Machinery & Electronics New Technic Institute, *SY42STH47-1504A High Torque Hybrid Stepping Motor Specifications*. Changzhou Songyang Machinery & Electronics New Technic Institute, 2014.
- [6] R. G. Budynas and J. K. Nisbett, *Shigley’s Mechanical Engineering Design*. New York: McGraw-Hill Education, 11th ed., 2019.
- [7] DraftShift Design, “Stealthchanger,” 2024.
- [8] International Organization for Standardization, “Parallel pins, of unhardened steel and austenitic stainless steel,” Standard ISO 2338:1997, International Organization for Standardization, Geneva, CH, 1997.
- [9] International Organization for Standardization, “Geometrical product specifications (gps) — iso code system for tolerances on linear sizes — part 1: Basis of tolerances, deviations and fits,” Standard ISO 286-1:2010, International Organization for Standardization, Geneva, CH, 2010.
- [10] G. Fantoni, M. Santochi, G. Dini, K. Tracht, B. Scholz-Reiter, J. Fleischer, T. K. Lien, G. Seliger, G. Reinhart, J. Franke, *et al.*, “Grasping devices and methods in automated production processes,” *CIRP annals*, vol. 63, no. 2, pp. 679–701, 2014.
- [11] G. J. Monkman, *Robot grippers*. Wiley Online Library, 2007.
- [12] A. Dietzel, “A lightweight suction gripper for micro assembly.”
- [13] SCHUNK GmbH & Co. KG, “EGP — Gripper for Small Components: Product Specification.” https://schunk.com/de/en/gripping-systems/parallel-gripper/egp/c/PGR_1593, 2024. [Online; accessed Apr. 2026].
- [14] O. A. Mohamed, S. H. Masood, and J. L. Bhowmik, “Optimization of fused deposition modeling process parameters: a review of current research and future prospects,” *Advances in manufacturing*, vol. 3, no. 1, pp. 42–53, 2015.
- [15] A. H. Slocum, *Precision machine design*. Society of Manufacturing Engineers, 1992.
- [16] R. Deimel and O. Brock, “A novel type of compliant and underactuated robotic hand for dexterous grasping,” *The International Journal of Robotics Research*, vol. 35, no. 1-3, pp. 161–185, 2016.
- [17] K. J. Astrom, “Pid controllers: theory, design, and tuning,” *The international society of measurement and control*, 1995.
- [18] ams AG, “AS5600 Programmable 12-Bit High-Resolution Contactless Magnetic Rotary Position Sensor.” <https://ams-osram.com>, 2018. Datasheet. [Online; accessed Apr. 2026].

- [19] e-con Systems, “What are global shutter and rolling shutter? how to choose the one that fits the application,” 2023. Accessed: 2026-04-15.
- [20] Basler AG, “ace aca2500-14gc gige camera,” 2026. Accessed: 2026-04-27.
- [21] e-con Systems, “Ar0234 global shutter camera module,” 2026. Accessed: 2026-04-27.
- [22] Intel Corporation, “Intel realsense depth camera d435,” 2026. Accessed: 2026-04-27.
- [23] Arducam, “Arducam 1.58mp imx296 color global shutter usb camera module,” 2026. Accessed: 2026-04-27.
- [24] Cognex Corporation, “In-sight 7000 series vision systems,” 2026. Accessed: 2026-04-27.
- [25] Raspberry Pi Ltd, “Raspberry pi global shutter camera,” 2026. Accessed: 2026-04-27.
- [26] A. Kumar, P. Kumar, R. K. Mittal, and H. Singh, “Chapter 6 - printing file formats for additive manufacturing technologies,” in *Advances in Additive Manufacturing* (A. Kumar, R. K. Mittal, and A. Haleem, eds.), Additive Manufacturing Materials and Technologies, pp. 87–102, Elsevier, 2023.
- [27] T. Loveland, “Understanding and writing g m code for cnc machines,” *Technology and Engineering Teacher*, vol. 71, pp. 24–29, Dec 2012.
- [28] Marlin Firmware, “Marlin firmware g-code index.” Accessed: 2026-04-23.
- [29] R. G. Budynas, J. K. Nisbett, *et al.*, *Shigley’s mechanical engineering design*, vol. 8. McGraw-Hill New York, 2008.
- [30] N. Stoimenov, M. Kandeve, M. Zagorski, and P. Panev, “Static and kinetic friction of 3d printed polymers and composites,” *Tribology in Industry*, vol. 46, no. 1, p. 97, 2024.
- [31] Tower Pro Co., Ltd., “SG90 Micro Servo Motor Datasheet.” <http://www.towerpro.com.tw>, 2026. [Online; accessed Apr. 2026].
- [32] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback control of dynamic systems*. Pearson., 2015.
- [33] ams AG, *AS5600 Magnetic Rotary Position Sensor Datasheet*, 2018. Accessed: 2026-04-02.
- [34] W. Kabsch, “A solution for the best rotation to relate two sets of vectors,” *Acta Crystallographica Section A*, vol. 32, no. 5, pp. 922–923.
- [35] The Pi Hut, “C-mount lens for raspberry pi hq camera (2.8–12mm varifocal),” 2026. Accessed: 2026-04-27.

8 Team Contribution

Rahul Jo-Shua Thanasekaran: Led the design, development, and integration of the machine vision subsystem for automated inspection and verification. This included implementation of a Raspberry Pi-based global shutter camera system, dataset collection and pre-processing, and development of a MobileNetV2 transfer learning model for nut placement classification. The model was optimised and deployed using TensorFlow Lite for embedded inference, and integrated with the wider system through communication between the Raspberry Pi and host controller for real-time triggering and result transmission.

Report Contribution: Sections 1, 2.3, 4.3, 5.3, 5.5 and 6.3.

Nikolaos Mitsokapas: Led the full development of the end-effector and wrist subsystem, encompassing end-effector type selection, iterative FDM design of the gripper fingers and bevel gear wrist assembly, AS5600 magnetic encoder integration, and complete Arduino PID firmware implementation including motor calibration, closed-loop wrist positioning, and gripper actuation. Closed-loop performance was characterised across fourteen angle transitions, confirming the M5 cavity insertion tolerance was met in twelve of fourteen tested cases.

Report Contribution: Sections 1, 2.2, 3.2, 4.2, 5.2, and 6.2.

Jaiveer Safri: Led the design, development, and mechanical integration of the independent, active tool-changing gantry and the gravity-reliant kinematic coupling. This included modifying the native Marlin firmware to define and control the custom axis and configuring the electromechanical gripper logic. Furthermore, quantitative tensile testing was conducted to evaluate the structural integrity and interlayer adhesion recovery of the 3D-printed components following mid-print automated embedding.

Report Contribution: Sections 2.1, 3.1, 4.1, 5.1, 5.1.3 and 6.1.

Nevain Dewage: Led the development and integration of the automated component embedding software pipeline. This included the implementation of the embedded component extraction script and the G-code streaming system was also developed to transmit instructions directly to the printer through serial communication. Also organised the communication between subsystems to ensure a seamless automated workflow.

Report Contribution: Sections 2.4, 3.3, 4.4, 5.4, 5.5, and 6.4.

9 Appendix

9.1 Appendix A: Machine Vision Implementation Code

This appendix presents the Python implementation of the MobileNetV2-based machine vision pipeline, including dataset preparation, model training, evaluation, and inference.

```
import os
import numpy as np
import tensorflow as tf
from tensorflow import keras
from PIL import Image
import matplotlib.pyplot as plt

DATA_ROOT = "/Users/rahuljo-shua/Desktop/Input"
TEST_DIR = "/Users/rahuljo-shua/Desktop/Test"
IMG_SIZE = (224, 224)
BATCH_SIZE = 16
EPOCHS = 40
MODEL_PATH = "/Users/rahuljo-shua/Desktop/placement_classifier_mobilenet_copy.
keras"
SEED = 123
THRESHOLD = 0.75

def load_dataset_splits():

    fail_files = []
    success_files = []

    for nut_size in os.listdir(DATA_ROOT):

        nut_path = os.path.join(DATA_ROOT, nut_size)

        if not os.path.isdir(nut_path):
            continue

        fail_dir = os.path.join(nut_path, "Fail")
        success_dir = os.path.join(nut_path, "Success")

        if os.path.exists(fail_dir):
            fail_files += [
                os.path.join(fail_dir, f)
                for f in os.listdir(fail_dir)
                if f.lower().endswith((".jpg", ".jpeg", ".png"))
            ]

        if os.path.exists(success_dir):
            success_files += [
                os.path.join(success_dir, f)
                for f in os.listdir(success_dir)
                if f.lower().endswith((".jpg", ".jpeg", ".png"))
            ]

    print(f"[INFO] Total FAIL images: {len(fail_files)}")
    print(f"[INFO] Total SUCCESS images: {len(success_files)}")

    all_files = np.array(fail_files + success_files)
    all_labels = np.array([0]*len(fail_files) + [1]*len(success_files))

    rng = np.random.default_rng(SEED)
    idx = rng.permutation(len(all_files))
```

```

all_files = all_files[idx]
all_labels = all_labels[idx]

split_index = int(0.8 * len(all_files))

train_files = all_files[:split_index]
train_labels = all_labels[:split_index]

val_files = all_files[split_index:]
val_labels = all_labels[split_index:]

print(f"[INFO] Training images: {len(train_files)}")
print(f"[INFO] Validation images: {len(val_files)}")

return train_files, train_labels, val_files, val_labels

def load_image(path, label):

    def _load(path):
        path = path.numpy().decode("utf-8")

        try:
            img = Image.open(path).convert("RGB")
            img = img.resize(IMG_SIZE)
            img = np.array(img).astype(np.float32)
        except:
            return np.zeros((IMG_SIZE[0], IMG_SIZE[1], 3), dtype=np.float32)

        return img

    img = tf.py_function(_load, [path], tf.float32)
    img.set_shape([IMG_SIZE[0], IMG_SIZE[1], 3])

    img = tf.keras.applications.mobilenet_v2.preprocess_input(img)

    return img, label

def build_datasets():

    train_files, train_labels, val_files, val_labels = load_dataset_splits()

    train_ds = tf.data.Dataset.from_tensor_slices((train_files, train_labels))
    train_ds = train_ds.shuffle(len(train_files))
    train_ds = train_ds.map(load_image, num_parallel_calls=tf.data.AUTOTUNE)
    train_ds = train_ds.batch(BATCH_SIZE)
    train_ds = train_ds.prefetch(tf.data.AUTOTUNE)

    val_ds = tf.data.Dataset.from_tensor_slices((val_files, val_labels))
    val_ds = val_ds.map(load_image, num_parallel_calls=tf.data.AUTOTUNE)
    val_ds = val_ds.batch(1)
    val_ds = val_ds.prefetch(tf.data.AUTOTUNE)

    return train_ds, val_ds, val_files, val_labels

def build_model():

    data_augmentation = keras.Sequential([
        keras.layers.RandomFlip("horizontal"),

```

```

        keras.layers.RandomRotation(0.15),
        keras.layers.RandomZoom(0.15),
        keras.layers.RandomBrightness(0.2),
        keras.layers.RandomContrast(0.2),
    ])

    base = keras.applications.MobileNetV2(
        input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3),
        include_top=False,
        weights="imagenet"
    )

    base.trainable = True
    for layer in base.layers[:-20]:
        layer.trainable = False

    model = keras.Sequential([
        data_augmentation,
        base,
        keras.layers.GlobalAveragePooling2D(),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dropout(0.4),
        keras.layers.Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(3e-5),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    model.summary()
    return model

def evaluate_validation_set(model, val_files, val_labels):

    print("\n===== VALIDATION RESULTS =====\n")

    correct = 0
    total = 0

    print(f"{'Filename':<25} {'Pred':<10} {'True':<10} {'Prob':<10} {'Result'}")
    print("")
    print("-" * 70)

    for path, true_label in zip(val_files, val_labels):

        fname = os.path.basename(path)
        true_text = "SUCCESS" if true_label == 1 else "FAIL"

        try:
            img = Image.open(path).convert("RGB")
            img = img.resize(IMG_SIZE)
            img = np.array(img).astype("float32")
        except:
            continue

        img = tf.keras.applications.mobilenet_v2.preprocess_input(img)
        batch = np.expand_dims(img, axis=0)

```

```

    prob = float(model.predict(batch, verbose=0)[0][0])
    pred_label = 1 if prob >= THRESHOLD else 0
    pred_text = "SUCCESS" if pred_label == 1 else "FAIL"

    is_correct = (pred_label == true_label)

    if is_correct:
        correct += 1

    total += 1

    print(f"{fname:<25} {pred_text:<10} {true_text:<10} {prob:<10.4f} {' ' if is_correct else ' '}")

if total > 0:
    accuracy = (correct / total) * 100
    print("-" * 70)
    print(f"Accuracy: {accuracy:.2f}% ({correct}/{total})")
    print("=====\n")

def inference_on_folder(model, folder_path):

    print(f"\n===== INFERENCE =====\n")

    for fname in os.listdir(folder_path):

        if not fname.lower().endswith((".jpg", ".jpeg", ".png")):
            continue

        path = os.path.join(folder_path, fname)

        try:
            img = Image.open(path).convert("RGB")
            img = img.resize(IMG_SIZE)
            img = np.array(img).astype("float32")
        except:
            print(f"{fname} SKIPPED")
            continue

        img = tf.keras.applications.mobilenet_v2.preprocess_input(img)
        batch = np.expand_dims(img, axis=0)

        prob = float(model.predict(batch, verbose=0)[0][0])
        pred = "SUCCESS" if prob >= THRESHOLD else "FAIL"

        print(f"{fname} {pred} (prob={prob:.4f})")

if __name__ == "__main__":

    print("\n===== Placement Classifier (MobileNetV2 Transfer Learning) =====")
    print("1 = Train model and save it")
    print("2 = Load saved model and run inference on TEST folder")
    print("=====\n")

    choice = input("Select option (1/2): ").strip()

    if choice == "1":

```

```

train_ds, val_ds, val_files, val_labels = build_datasets()
model = build_model()

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

print("\n[INFO] Training model...\n")
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=[early_stop],
    class_weight={0: 1.5, 1: 1.0}
)

plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.grid()
plt.savefig("/Users/rahuljo-shua/Desktop/accuracy_plot.png")
plt.show()

plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Training vs Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.grid()
plt.savefig("/Users/rahuljo-shua/Desktop/loss_plot.png")
plt.show()

print("\n[INFO] Validation results:")
evaluate_validation_set(model, val_files, val_labels)

model.save(MODEL_PATH)
print(f"\n[SUCCESS] Model saved to {MODEL_PATH}\n")

elif choice == "2":

    if not os.path.exists(MODEL_PATH):
        print("[ERROR] Train a model first (option 1).")
        exit()

    model = keras.models.load_model(MODEL_PATH)
    inference_on_folder(model, TEST_DIR)

else:
    print("[ERROR] Invalid option.")

```

9.2 Appendix B: Embedded Component Extraction Code

```

import zipfile
import xml.etree.ElementTree as ET
import io
import json
import numpy as np
import time
from pathlib import Path

#Change to the user-input CAD file path
input_file = r"C:/Users/nevai/OneDrive/Desktop/UCL/Year 4/Project 2/Export.3mf"
"
#Change to the required print CAD output file path
output_file = r"C:/Users/nevai/OneDrive/Desktop/UCL/Year 4/Project 2/Print.3mf"
"
#Change to the required embedded components information file path
embed_data_file = r"C:/Users/nevai/OneDrive/Desktop/UCL/Year 4/Project 2/
    embedded_components.json"
#Change to the component library path
LIBRARY_FOLDER = Path(r"C:/Users/nevai/OneDrive/Desktop/UCL/Year 4/Project 2/
    component_library")

TARGET_CENTER_X = 150.0
TARGET_CENTER_Y = 150.0
TARGET_MIN_Z = 0.0

#Timer for system efficiency check
start_time = time.perf_counter()

NS = "http://schemas.microsoft.com/3dmanufacturing/core/2015/02"
ns = {"m": NS}
ET.register_namespace("", NS)

def rotation_matrix_to_euler_zyx(R):
    """Convert rotation matrix to Euler angles."""
    sy = np.sqrt(R[0, 0] ** 2 + R[1, 0] ** 2)
    singular = sy < 1e-9

    if not singular:
        yaw = np.arctan2(R[1, 0], R[0, 0])
        pitch = np.arctan2(-R[2, 0], sy)
        roll = np.arctan2(R[2, 1], R[2, 2])
    else:
        yaw = np.arctan2(-R[0, 1], R[1, 1])
        pitch = np.arctan2(-R[2, 0], sy)
        roll = 0.0

    return float(np.degrees(yaw)), float(np.degrees(pitch)), float(np.degrees(
        roll))

def kabsch_rotation(A, B):
    """Compute rotation matrix aligning mesh B to mesh A."""
    if A.shape != B.shape:
        raise ValueError("Vertex count mismatch between canonical and
            extracted mesh.")

    cA = A.mean(axis=0)
    cB = B.mean(axis=0)

    AA = A - cA
    BB = B - cB

```

```

H = BB.T @ AA
U, _, Vt = np.linalg.svd(H)
R = Vt.T @ U.T

if np.linalg.det(R) < 0:
    Vt[2, :] *= -1
    R = Vt.T @ U.T

return R

def load_vertices_from_3mf(filepath):
    """Load vertex coordinates from a 3MF file."""
    if not filepath.exists():
        raise FileNotFoundError(f"Library component not found: {filepath}")

    with zipfile.ZipFile(filepath, 'r') as z:
        model_xml = z.read("3D/3dmodel.model")

    tree = ET.ElementTree(ET.fromstring(model_xml))
    root = tree.getroot()

    verts = []
    for v in root.findall("./m:vertex", ns):
        verts.append([
            float(v.get("x", 0)),
            float(v.get("y", 0)),
            float(v.get("z", 0))
        ])

    if len(verts) < 3:
        raise ValueError(f"No vertices found in {filepath}")

    return np.array(verts)

def compute_bbox(pts):
    """Compute bounding box, centre, and size of a mesh."""
    min_vals = pts.min(axis=0)
    max_vals = pts.max(axis=0)
    centre = (min_vals + max_vals) / 2.0
    size = max_vals - min_vals
    return centre, min_vals, max_vals, size

def apply_translation_to_xml_vertices(root, dx, dy, dz):
    """Apply translation to all vertices in XML."""
    if dx == 0 and dy == 0 and dz == 0:
        return

    for v in root.findall("./m:vertex", ns):
        x = float(v.get("x", 0))
        y = float(v.get("y", 0))
        z = float(v.get("z", 0))
        v.set("x", f"{x + dx:.6f}")
        v.set("y", f"{y + dy:.6f}")
        v.set("z", f"{z + dz:.6f}")

def compute_translation_for_print_model(root, embed_ids):

```



```

    """Compute translation to centre and align print model."""
    objects = {obj.get("id"): obj for obj in root.findall("./m:object", ns)}
    build_elem = root.find("./m:build", ns)

    min_x = min_y = min_z = float("inf")
    max_x = max_y = max_z = float("-inf")

    for item in build_elem.findall("m:item", ns):
        oid = item.get("objectid")
        if oid in embed_ids:
            continue

        obj = objects.get(oid)
        if obj is None:
            continue

        name = obj.get("name", "") or ""
        if name != "Print_Model":
            continue

        for v in obj.findall("./m:vertex", ns):
            x = float(v.get("x", 0))
            y = float(v.get("y", 0))
            z = float(v.get("z", 0))

            min_x = min(min_x, x)
            max_x = max(max_x, x)
            min_y = min(min_y, y)
            max_y = max(max_y, y)
            min_z = min(min_z, z)
            max_z = max(max_z, z)

        cx = (min_x + max_x) / 2.0
        cy = (min_y + max_y) / 2.0

        dx = TARGET_CENTER_X - cx
        dy = TARGET_CENTER_Y - cy
        dz = TARGET_MIN_Z - min_z

    return dx, dy, dz, {
        "min_x": min_x,
        "max_x": max_x,
        "min_y": min_y,
        "max_y": max_y,
        "min_z": min_z,
        "max_z": max_z
    }

}

with zipfile.ZipFile(input_file, "r") as zin:
    file_data = {name: zin.read(name) for name in zin.namelist()}

model_path = "3D/3dmodel.model"
tree = ET.ElementTree(ET.fromstring(file_data[model_path]))
root = tree.getroot()

objects = {obj.get("id"): obj for obj in root.findall("./m:object", ns)}
build_elem = root.find("./m:build", ns)

embedded_components_info = []
embed_ids = []

```

```

#Isolate and save embedded components

for item in build_elem.findall("m:item", ns):
    oid = item.get("objectid")
    obj = objects.get(oid)
    if obj is None:
        continue

    name = obj.get("name", "") or ""
    if not name.startswith("Embed-"):
        continue

    clean_name = name.replace("Embed-", "", 1)
    embed_ids.append(oid)

    print(f'Component "{clean_name}" detected')

    verts = []
    for v in obj.findall("./m:vertex", ns):
        verts.append([
            float(v.get("x", 0)),
            float(v.get("y", 0)),
            float(v.get("z", 0))
        ])

    extracted_pts = np.array(verts)

    centre, min_vals, max_vals, size = compute_bbox(extracted_pts)

    library_file = LIBRARY_FOLDER / f"{clean_name}.3mf"
    canonical_pts = load_vertices_from_3mf(library_file)

    R = kabsch_rotation(canonical_pts, extracted_pts)
    yaw, pitch, roll = rotation_matrix_to_euler_zyx(R)

    yaw = round(yaw / 2.0) * 2.0
    pitch = round(pitch / 2.0) * 2.0
    roll = round(roll / 2.0) * 2.0

    embedded_components_info.append({
        "id": oid,
        "name": clean_name,
        "centre": {
            "x": float(centre[0]),
            "y": float(centre[1]),
            "z": float(centre[2])
        },
        "bbox": {
            "min": {"x": float(min_vals[0]), "y": float(min_vals[1]), "z":
                float(min_vals[2])},
            "max": {"x": float(max_vals[0]), "y": float(max_vals[1]), "z":
                float(max_vals[2])},
            "size": {"x": float(size[0]), "y": float(size[1]), "z": float(size
                [2])}
        },
        "min_z": float(min_vals[2]),
        "max_z": float(max_vals[2]),
        "rotation_matrix": R.tolist(),
        "orientation": {
            "yaw_deg": yaw,

```

```

        "pitch_deg": pitch,
        "roll_deg": roll
    }
})

print(f'Component "{clean_name}" extracted')

#Remove embedded components

resources = root.find("./m:resources", ns)
if resources is not None:
    for obj in list(resources.findall("m:object", ns)):
        if obj.get("id") in embed_ids:
            resources.remove(obj)

for item in list(build_elem.findall("m:item", ns)):
    if item.get("objectid") in embed_ids:
        build_elem.remove(item)

# Centre print model

dx, dy, dz, print_bounds = compute_translation_for_print_model(root, set(
    embed_ids))

apply_translation_to_xml_vertices(root, dx, dy, dz)

for comp in embedded_components_info:
    comp["centre"]["x"] += dx
    comp["centre"]["y"] += dy
    comp["centre"]["z"] += dz
    comp["min_z"] += dz
    comp["max_z"] += dz

end_time = time.perf_counter()
execution_time = end_time - start_time

# Store updated files
model_bytes = io.BytesIO()
tree.write(model_bytes, encoding="utf-8", xml_declaration=True)
file_data[model_path] = model_bytes.getvalue()

with zipfile.ZipFile(output_file, "w", compression=zipfile.ZIP_DEFLATED) as
    zout:
        for name, data in file_data.items():
            zout.writestr(name, data)

output_data = {
    "print_model": {
        "name": "Print_Model",
        "bbox": {
            "min": {
                "x": float(print_bounds["min_x"] + dx),
                "y": float(print_bounds["min_y"] + dy),
                "z": float(print_bounds["min_z"] + dz)
            },
            "max": {
                "x": float(print_bounds["max_x"] + dx),
                "y": float(print_bounds["max_y"] + dy),
                "z": float(print_bounds["max_z"] + dz)
            }
        }
    },
    },

```

```

        "max_x": float(print_bounds["max_x"] + dx),
        "max_y": float(print_bounds["max_y"] + dy)
    },
    "embedded_components": embedded_components_info,
    "performance": {
        "execution_time_s": float(execution_time),
        "number_of_embedded_components": int(len(embedded_components_info))
    }
}

with open(embed_data_file, "w") as f:
    json.dump(output_data, f, indent=4)

print("Complete!")
print(f"Extracted {len(embedded_components_info)} components.")
print(f"Execution time: {execution_time:.6f} s")

```

9.3 Appendix C: G-code Streaming Implementation Script

```

from math import floor
from pathlib import Path
import json
import serial
import time

#Change to embedded component detail file path
EMBED_JSON = Path(r"C:\Users\nevai\OneDrive\Desktop\UCL\Year 4\Project 2\
    embedded_components.json")
#Change to g-code file path output from slicer
INPUT_GCODE = Path(r"C:\Users\nevai\OneDrive\Desktop\UCL\Year 4\Project 2\
    Print_T5-SINGLE.gcode")

SERIAL_PORT = "COM7"
BAUDRATE = 250000
SERIAL_PORT_G = "COM3"
BAUDRATE_G = 115200
TIMEOUT = 10

#Set to true for manual embedding
MANUAL_EMBED = False

def load_components_geometry(json_path: Path):
    """Load embedded component geometry and print model data from JSON."""
    if not json_path.exists():
        print("No embedding file found      running in print-only mode.")
        return None, None
    with open(json_path, "r") as f:
        data = json.load(f)

    print_model = data.get("print_model", {})
    components = data.get("embedded_components", [])
    if not components:
        print("Embedding file contains no embedded components      running in
            print-only mode.")

    results = []
    for comp in components:
        name = comp.get("name", "(unnamed)")
        centre = comp.get("centre", {})
        orientation = comp.get("orientation", {})

```

```

        results.append({
            "name": name,
            "top_z": float(comp.get("max_z", 0.0)),
            "bottom_z": float(comp.get("min_z", 0.0)),
            "mid_x": float(centre.get("x", 0.0)),
            "mid_y": float(centre.get("y", 0.0)),
            "yaw_deg": float(orientation.get("yaw_deg", 0.0)),
            "pitch_deg": float(orientation.get("pitch_deg", 0.0)),
            "roll_deg": float(orientation.get("roll_deg", 0.0)),
        })
    print(f"Loaded {len(results)} embedded component(s).")
    return results, print_model

def extract_layer_height_from_gcode(gcode_path: Path):
    """Extract layer height from G-code file."""
    z_values = []
    with open(gcode_path, "r") as f:
        for line in f:
            line = line.strip()
            if line.lower().startswith(";layer height:"):
                return float(line.split(":")[1].strip())
            if line.startswith(";Z:"):
                try:
                    z_values.append(float(line.split(":")[1]))
                except ValueError:
                    pass

    if len(z_values) >= 2:
        return round(z_values[1] - z_values[0], 5)
    raise ValueError("Could not determine layer height.")

def connect_printer(port, baud):
    """Establish serial connection to printer."""
    try:
        ser = serial.Serial(port, baud, timeout=2, write_timeout=2)
    except serial.SerialException as e:
        print(f"Failed to open serial: {e}")
        return None

    time.sleep(2)
    ser.reset_input_buffer()
    ser.reset_output_buffer()
    return ser

def _send_line(ser, line: str):
    """Send input line to serial device."""
    ser.write((line.strip() + "\n").encode("ascii"))
    ser.flush()

def send_gcode(ser, line: str):
    """Send G-code command and wait for acknowledgement."""
    stripped = line.split(";")[0].strip()
    if not stripped:
        return

    cmd = stripped.split()[0].upper()
    _send_line(ser, stripped)

    if cmd == "G28":
        send_gcode(ser, "M400")
        return

```

```

if cmd in {"M109", "M190"}:
    timeout = 300
else:
    timeout = 5

start = time.time()
last_activity = start
while True:
    if time.time() - last_activity > timeout:
        raise TimeoutError(f"No OK for: {line}")
    resp = ser.readline().decode(errors="ignore").strip()
    if not resp:
        continue

    last_activity = time.time()
    if resp.lower().startswith("ok"):
        return
    if resp.lower().startswith("error"):
        raise RuntimeError(resp)

def send_and_read(ser, line: str, timeout=5):
    """Send command and return all responses until OK."""
    stripped = line.split(";", 1)[0].strip()
    if not stripped:
        return []
    _send_line(ser, stripped)

    responses = []
    start = time.time()
    last_activity = start
    while True:
        if time.time() - last_activity > timeout:
            raise TimeoutError(f"No OK for: {line}")
        resp = ser.readline().decode(errors="ignore").strip()
        if not resp:
            continue
        last_activity = time.time()

        if resp.lower().startswith("ok"):
            return responses
        if resp.lower().startswith("error"):
            raise RuntimeError(resp)
        responses.append(resp)

def send_command(ser, command):
    """Send command without response handling."""
    cmd = command + "\n"
    print(f">>> Sending: {command}")
    ser.write(cmd.encode("utf-8"))
    time.sleep(0.2)

    while ser.in_waiting:
        response = ser.readline().decode("utf-8").strip()
        print(f"<<< {response}")

def initialise_printer(ser):
    """Initialise printer settings and communication."""
    info = send_and_read(ser, "M115")
    for line in info:

```

```

        print("FW:", line)
    send_gcode(ser, "G90")
    send_gcode(ser, "G21")

def perform_embedding_sequence(ser, component, outer_wall_moves, prev_layer_z,
    current_layer_z):
    """Execute embedding sequence including pause, reheating, and purge."""

    SAFE_LIFT = 15
    HOLD_TEMP = 180
    REHEAT_TEMP = 190
    PRINT_TEMP = 205
    HOVER_OFFSET = 0.3

    send_gcode(ser, "M400")
    send_gcode(ser, "M106 S0")
    send_gcode(ser, f"M104 S{HOLD_TEMP}")
    send_gcode(ser, "G91")
    send_gcode(ser, f"G1 Z{SAFE_LIFT} F3000")
    send_gcode(ser, "G90")

    if MANUAL_EMBED:
        print("\n=== EMBEDDING PAUSE ===")
        print(f"Insert component: {component['name']}")
        print(f"Target position (CAD): X={component['mid_x']:.2f}, Y={
            component['mid_y']:.2f}")
        input("Press ENTER to continue...")
        send_gcode(ser, "M400")
    else:
        send_gcode(ser, "G4 P10000")

    #Reheat embedding layer
    if outer_wall_moves and prev_layer_z is not None:
        send_gcode(ser, f"M109 S{REHEAT_TEMP}")
        send_gcode(ser, "M83")
        hover_z = prev_layer_z + HOVER_OFFSET
        send_gcode(ser, f"G1 Z{hover_z:.3f} F1500")
        send_gcode(ser, "G1 F900")
        for move in outer_wall_moves:
            send_gcode(ser, move)
        send_gcode(ser, "G91")
        send_gcode(ser, f"G1 Z{SAFE_LIFT} F2000")
        send_gcode(ser, "G90")

    send_gcode(ser, f"M109 S{PRINT_TEMP}")
    send_gcode(ser, "M400")
    send_gcode(ser, "G90")
    send_gcode(ser, "M83")
    send_gcode(ser, f"G1 Z{current_layer_z:.3f} F1500")
    send_gcode(ser, "M83")
    send_gcode(ser, "M400")

def extract_layer(component, layer_height):
    """Compute embedding layer index for a component."""
    return max(0, floor(component["top_z"] / layer_height) - 1)

def stream_print(gcode_path, components, ser):
    """Stream G-code and trigger embedding at specified layers."""
    layer_height = extract_layer_height_from_gcode(gcode_path)
    print(f"Layer height: {layer_height}")

```

```

embed_layers = {}
if components:
    for comp in components:
        embed_layers.setdefault(
            extract_layer(comp, layer_height), []
        ).append(comp)

current_layer = -1
outer_wall_moves = []
last_outer_wall_moves = []
in_outer_wall = False
current_layer_z = None
last_layer_z = None
captured_z = False
embedding_pending = False

with open(gcode_path, "r") as f:
    for line in f:
        stripped = line.strip()
        if stripped.startswith("; LAYER:"):
            last_outer_wall_moves = outer_wall_moves.copy()
            current_layer = int(stripped.split(":")[1])
            outer_wall_moves = []
            in_outer_wall = False
            captured_z = False
            embedding_pending = current_layer in embed_layers

        if (not captured_z
            and stripped.startswith(("G0", "G1"))
            and "Z" in stripped):
            last_layer_z = current_layer_z
            for part in stripped.split():
                if part.startswith("Z"):
                    current_layer_z = float(part[1:])
                    break

            captured_z = True
            if embedding_pending:
                send_gcode(ser, line)
                for comp in embed_layers[current_layer]:
                    perform_embedding_sequence(
                        ser,
                        comp,
                        last_outer_wall_moves,
                        last_layer_z,
                        current_layer_z)
                del embed_layers[current_layer]
                embedding_pending = False
                continue

        if stripped.startswith(";TYPE:Outer wall"):
            in_outer_wall = True
            continue

        if stripped.startswith(";TYPE:") and not stripped.startswith(";
TYPE:Outer wall"):
            in_outer_wall = False

        if in_outer_wall and stripped.startswith("G1") and "X" in stripped
            and "Y" in stripped:
            clean = stripped.split("E")[0].split("F")[0].strip()
            outer_wall_moves.append(clean)
        send_gcode(ser, line)

```



```

def main():
    """Run full printing and embedding pipeline."""
    components, print_model = load_components_geometry(EMBED_JSON)
    ser = connect_printer(SERIAL_PORT, BAUDRATE)
    if not ser:
        return
    try:
        initialise_printer(ser)
        stream_print(INPUT_GCODE, components, ser)
    except Exception as e:
        print("ERROR:", e)
    try:
        send_gcode(ser, "MO")
    except Exception:
        pass
    finally:
        ser.close()

if __name__ == "__main__":
    main()

```

9.4 Appendix D: End-effector Arduino Control Code

The following listing presents the complete Arduino UNO firmware for the end-effector subsystem. The code implements serial command reception at 115200 baud, AS5600 magnetic encoder reading via I2C, PID-based closed-loop wrist yaw positioning with motor-specific PWM calibration functions, and open-loop gripper actuation via a nut-size lookup table. An acknowledgment is issued over serial once the measured wrist angle has remained within $\pm 0.5^\circ$ of the commanded target for five consecutive control loop iterations at a 50 ms period.

```

#include <Wire.h>
#include <Servo.h>
#include "NutAngles.h"

const int YAW_SERVO1_PIN = 9;
const int YAW_SERVO2_PIN = 10;
const int GRIPPER_SERVO_PIN = 11;

double dt, last_time;
double integral = 0.0;
double previous = 0.0;
double output = 0.0;

double Kp = 0.3;
double Ki = 0.4;
double Kd = 0.001;

Servo yawServo1;
Servo yawServo2;
Servo gripperServo;

// ANGLE STATE
double desiredAngle = 0.0;
double targetAngle = 0.0;
double currentAngle = 0.0;

const double RAMP_STEP = 0.8;
bool commandReceived = false;

// AS5600 STATE

```

```

float degAngle = 0.0;
float startAngle = 0.0;
float correctedAngle = 0.0;

// OK SIGNAL STATE
float previousAngleForSettle = 0.0; // last angle reading for comparison
bool okSent = false; // prevents sending OK multiple times
const float SETTLE_THRESHOLD = 0.5; // degrees - if change less than this,
    settled
const float REACH_TOLERANCE = 2.0; // degrees - must be within this of desired

int angleToPWM_S1(double angle) {
    double pwm = (angle + 0.299414) / 0.013198 + 500.0;
    return (int)constrain(pwm, 500, 2500);
}

int angleToPWM_S2(double angle) {
    double pwm = (angle + 0.589778) / 0.013278 + 500.0;
    return (int)constrain(pwm, 500, 2500);
}

void writeYaw(double totalAngle) {
    totalAngle = constrain(totalAngle, 0, 180);
    double half = totalAngle / 2.0;
    yawServo1.writeMicroseconds(angleToPWM_S1(half));
    yawServo2.writeMicroseconds(angleToPWM_S2(half));
}

double pid(double error) {
    double proportional = error;
    integral += error * dt;
    integral = constrain(integral, -20, 20);
    double derivative = (error - previous) / dt;
    previous = error;
    return (Kp * proportional) + (Ki * integral) + (Kd * derivative);
}

// NUT LOOKUP
bool getNutAngles(int nutNumber, float &openAngle, float &closeAngle) {
    switch (nutNumber) {
        case 1: openAngle = M2_OPEN; closeAngle = M2_CLOSE; return true;
        case 2: openAngle = M2_5_OPEN; closeAngle = M2_5_CLOSE; return true;
        case 3: openAngle = M3_OPEN; closeAngle = M3_CLOSE; return true;
        case 4: openAngle = M3_5_OPEN; closeAngle = M3_5_CLOSE; return true;
        case 5: openAngle = M4_OPEN; closeAngle = M4_CLOSE; return true;
        case 6: openAngle = M5_OPEN; closeAngle = M5_CLOSE; return true;
        default: return false;
    }
}

void parseCommand(String cmd) {
    cmd.trim();

    // YAW command
    if (cmd.startsWith("YAW ")) {
        double angle = cmd.substring(4).toFloat();
        angle = constrain(angle, 0, 180);

        if (!commandReceived) {
            yawServo1.attach(YAW_SERVO1_PIN, 500, 2500);
            yawServo2.attach(YAW_SERVO2_PIN, 500, 2500);
        }
    }
}

```

```

    }

    desiredAngle = angle;
    integral = 0.0;
    commandReceived = true;
    okSent = false; // reset OK flag for new command
    previousAngleForSettle = currentAngle; // reset settle comparison

    Serial.print("YAW command: ");
    Serial.println(angle, 1);
    return;
}

// NUT command
if (cmd.startsWith("NUT ")) {
    int firstSpace = cmd.indexOf(' ');
    int secondSpace = cmd.indexOf(' ', firstSpace + 1);

    if (firstSpace == -1 || secondSpace == -1) {
        Serial.println("ERR: format is NUT OPEN/CLOSE <1-6>");
        return;
    }

    String action = cmd.substring(firstSpace + 1, secondSpace);
    int nutNumber = cmd.substring(secondSpace + 1).toInt();
    action.trim();
    action.toUpperCase();

    float openAngle, closeAngle;
    if (!getNutAngles(nutNumber, openAngle, closeAngle)) {
        Serial.print("ERR: Unknown nut number: ");
        Serial.println(nutNumber);
        return;
    }

    if (action == "OPEN") {
        gripperServo.write(openAngle);
        Serial.print("Gripper OPEN for nut ");
        Serial.println(nutNumber);
        Serial.println("OK"); // gripper is direct - no feedback needed
    } else if (action == "CLOSE") {
        gripperServo.write(closeAngle);
        Serial.print("Gripper CLOSE for nut ");
        Serial.println(nutNumber);
        Serial.println("OK"); // gripper is direct - no feedback needed
    } else {
        Serial.println("ERR: action must be OPEN or CLOSE");
    }
    return;
}

Serial.print("ERR: Unknown command: ");
Serial.println(cmd);
}

// SETTLE CHECK
void checkSettleAndSendOK() {
    if (okSent) return;
    if (!commandReceived) return;

```

```

// Condition 1 - must have reached the target range
bool withinRange = abs(currentAngle - desiredAngle) <= REACH_TOLERANCE;

// Condition 2 - must have stopped moving
bool settled = abs(currentAngle - previousAngleForSettle) < SETTLE_THRESHOLD
;

if (withinRange && settled) {
    Serial.println("OK");
    okSent = true;
}

// Update previous angle for next comparison
previousAngleForSettle = currentAngle;
}

void setup() {
    Serial.begin(115200);
    Wire.begin();
    Wire.setClock(400000L);

    gripperServo.attach(GRIPPER_SERVO_PIN, 500, 2500);
    gripperServo.write(M2_CLOSE);

    checkMagnetPresence();
    ReadRawAngle();
    startAngle = degAngle;

    last_time = millis();

    Serial.println("Ready.");
    Serial.println("Commands: YAW <angle> | NUT OPEN <1-5> | NUT CLOSE <1-5>");
    Serial.println("Nuts: 1=M2 2=M2.5 3=M3 4=M3.5 5=M4 6=M5");
}

void loop() {

    // Read UART command
    if (Serial.available()) {
        String cmd = Serial.readStringUntil('\n');
        parseCommand(cmd);
    }

    // PID dt
    double now = millis();
    dt = (now - last_time) / 1000.0;
    last_time = now;
    if (dt <= 0) return;

    // Ramp targetAngle toward desiredAngle
    if (targetAngle < desiredAngle)
        targetAngle = min(targetAngle + RAMP_STEP, desiredAngle);
    else if (targetAngle > desiredAngle)
        targetAngle = max(targetAngle - RAMP_STEP, desiredAngle);

    // Read encoder
    ReadRawAngle();
    correctAngle();
    currentAngle = correctedAngle;
}

```

```

// PID
double error = targetAngle - currentAngle;
output = pid(error);
double commandAngle = constrain(targetAngle + output, 0, 180);

// Write yaw      only after first command received
if (commandReceived) writeYaw(commandAngle);

// Check if yaw has settled and send OK to printer if so
checkSettleAndSendOK();

delay(20);
}

void ReadRawAngle() {
  Wire.beginTransmission(0x36);
  Wire.write(0x0D);
  Wire.endTransmission();
  Wire.requestFrom(0x36, 1);
  while (Wire.available() == 0);
  int lowbyte = Wire.read();

  Wire.beginTransmission(0x36);
  Wire.write(0x0C);
  Wire.endTransmission();
  Wire.requestFrom(0x36, 1);
  while (Wire.available() == 0);
  word highbyte = Wire.read();

  highbyte = highbyte << 8;
  int rawAngle = highbyte | lowbyte;
  degAngle = rawAngle * 0.087890625;
}

void correctAngle() {
  correctedAngle = degAngle - startAngle;
  if (correctedAngle < 0) correctedAngle += 360;
  if (correctedAngle > 360) correctedAngle -= 360;
  if (correctedAngle > 270.0) correctedAngle = correctedAngle - 360.0;
}

void checkMagnetPresence() {
  int magnetStatus = 0;
  while ((magnetStatus & 32) != 32) {
    magnetStatus = 0;
    Wire.beginTransmission(0x36);
    Wire.write(0x0B);
    Wire.endTransmission();
    Wire.requestFrom(0x36, 1);
    while (Wire.available() == 0);
    magnetStatus = Wire.read();
    if (!(magnetStatus & 32)) {
      Serial.println("Waiting for magnet...");
      delay(500);
    }
  }
  Serial.println("Magnet found.");
  delay(500);
}

```



```

servo[0].attach(0);
servo[1].attach(0);

if (filteredAngle < 0.0) {
    filteredAngle = correctedAngle;
} else {
    filteredAngle = correctedAngle;
}

double current_ramp_angle = correctedAngle;
integral = 0.0;
previous = filteredAngle;
last_micros = micros() - 20000;

uint32_t start_time = millis();
uint32_t ramp_end_time = 0;

while (millis() - start_time < 25000) {
    TERN_(USE_WATCHDOG, hal.watchdog_refresh());

    uint32_t now_micros = micros();
    dt = (double)(now_micros - last_micros) / 1000000.0;
    if (dt < 0.02) {
        continue;
    }
    last_micros = now_micros;

    // Sweeps at exactly 25 degrees per second (0.5 / 0.02).
    double step = (200.0) * dt;
    if (current_ramp_angle < target_angle)
        current_ramp_angle = min(current_ramp_angle + step, (double)target_angle);
    else if (current_ramp_angle > target_angle)
        current_ramp_angle = max(current_ramp_angle - step, (double)target_angle);

    // Gracefully handle I2C dropouts from Stepper EMI
    if (!ReadRawAngle()) {
        // If the sensor packet fails, instantly loop back and try again next
        // microsecond.
        // This prevents shoving frozen data into the PID controller and
        // causing D-spikes!
        continue;
    }
    correctAngle();

    filteredAngle = (0.2 * correctedAngle) + (0.8 * filteredAngle);

    double currentAngle = filteredAngle;
    double error = current_ramp_angle - currentAngle;
    double output = pid(error, currentAngle);
    double commandAngle = constrain(current_ramp_angle + output, 0, 180);

    writeYaw(commandAngle);

    if (abs((double)target_angle - current_ramp_angle) < 0.1) {
        if (ramp_end_time == 0) ramp_end_time = millis();

        if (abs((double)target_angle - currentAngle) < 1.5 || (millis() -
            ramp_end_time > 3000)) {

```

```

        delay(10);
        break;
    }
}
}
}

```

9.6.2 2. RAMBo Pin Mapping for Additional U-Axis

File Location: Marlin/src/pins/rambo/pins_RAMBO.h

To support the extra U-axis stepper motor, the MX3 extension header on the RAMBo board was repurposed. The original MAX endstop pins and case light pins were undefined and reassigned to the step, direction, enable, and minimum endstop functions of the custom "I" axis (which translates to the U-axis in G-code).

Listing 2: RAMBo Pin Reassignment

```

// Add to the bottom of Marlin/src/pins/rambo/pins_RAMBO.h

// Map MX3 Extension Header to Custom I-Axis
#undef X_MAX_PIN
#undef Y_MAX_PIN
#undef Z_MAX_PIN
#define X_MAX_PIN -1
#define Y_MAX_PIN -1
#define Z_MAX_PIN -1

#undef CASE_LIGHT_PIN
#define CASE_LIGHT_PIN -1

#define I_STEP_PIN    30 // PC7 (was Z-MAX)
#define I_DIR_PIN     46 // PL3 (was CASE LIGHT)
#define I_ENABLE_PIN  24 // PA2 (was X-MAX)
#define I_MIN_PIN     23 // PA1 (was Y-MAX)

```

9.6.3 3. Configuration Limits and Kinematics

File Location: Marlin/Configuration.h

The physical workspace dimensions were updated to reflect the actual bounds of the machine, including a critical override for the Y-axis maximum position to accommodate the new toolhead/gripper geometry. Additionally, the U-axis stepper driver was configured as a TMC2209 standalone driver.

Listing 3: Kinematics and Workspace Limits

```

// Modify around line 2190 in Marlin/Configuration.h

// The size of the printable area
#define X_BED_SIZE 289.4 // <-- changed
#define Y_BED_SIZE 276.4 // <-- changed

// Travel limits (linear=mm, rotational=deg) after homing.
#define X_MIN_POS 0
#define Y_MIN_POS 0
#define Z_MIN_POS 0
#define X_MAX_POS 299.5 // <-- changed
#define Y_MAX_POS 322.4 // <-- changed 277.4 + 45

```



```

#define Z_MAX_POS 250    // <-- changed
#define I_MIN_POS 0
#define I_MAX_POS 120

// Modify around line 180 to define the custom axis driver
#define I_DRIVER_TYPE TMC2209_STANDALONE

// Modify around line 210 to map the internal "I" axis to the G-Code "U" axis
#ifdef I_DRIVER_TYPE
    #define AXIS4_NAME 'U' // :['A', 'B', 'C', 'U', 'V', 'W']
#endif

```

9.6.4 4. Advanced Configuration for Gripper Servos

File Location: Marlin/Configuration_adv.h

The specific servo pins for controlling the gripper's dual yaw servos and the main opening/closing mechanism were defined.

Listing 4: Gripper Servo Pin Definitions

```

// Modify the Gripper Settings section in Marlin/Configuration_adv.h

// Gripper Settings
#define GRIPPER_GCODE
#if ENABLED(GRIPPER_GCODE)
    #define GRIPPER_YAW_SERVO_PIN_1 4 // PG5
    #define GRIPPER_YAW_SERVO_PIN_2 5 // PE3
    #define GRIPPER_OPEN_SERVO_PIN 22 // MX1
#endif

```